3.3.6 Applying Q^H

In a future chapter, we will see that the QR factorization is used to solve the linear least-squares problem. To do so, we need to be able to compute $\hat{y} = Q^H y$ where $Q^H = H_{n-1} \cdots H_0$.

Let us start by computing $H_0 y$:

$$\begin{array}{l} \left(I - \frac{1}{\tau_1} \begin{pmatrix} 1 \\ u_2 \end{pmatrix} \begin{pmatrix} 1 \\ u_2 \end{pmatrix}^H \right) \begin{pmatrix} \psi_1 \\ y_2 \end{pmatrix} \\ = \\ \begin{pmatrix} \psi_1 \\ y_2 \end{pmatrix} - \begin{pmatrix} 1 \\ u_2 \end{pmatrix} \underbrace{\begin{pmatrix} 1 \\ u_2 \end{pmatrix}^H \begin{pmatrix} \psi_1 \\ y_2 \end{pmatrix} / \tau_1}_{\omega_1} \\ = \\ \begin{pmatrix} \psi_1 \\ y_2 \end{pmatrix} - \omega_1 \begin{pmatrix} 1 \\ u_2 \end{pmatrix} \\ = \\ \begin{pmatrix} \psi_1 - \omega_1 \\ y_2 - \omega_1 u_2 \end{pmatrix}. \end{array}$$

More generally, let us compute $H_k y$:

$$\left(I - \frac{1}{\tau_1} \begin{pmatrix} 0 \\ 1 \\ u_2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ u_2 \end{pmatrix}^H \right) \begin{pmatrix} y_0 \\ \psi_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} y_0 \\ \psi_1 - \omega_1 \\ y_2 - \omega_1 u_2 \end{pmatrix},$$

where $\omega_1 = (\psi_1 + u_2^H y_2)/\tau_1$. This motivates the algorithm in Figure 3.3.6.1 for computing $y := H_{n-1} \cdots H_0 y$ given the output matrix A and vector t from routine HQR_unb_var1.

$[y] = \text{Apply\_QH}(A, t, y)$

$A \rightarrow \begin{pmatrix} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{pmatrix}, t \rightarrow \begin{pmatrix} t_T \\ \hline t_B \end{pmatrix}, y \rightarrow \begin{pmatrix} y_T \\ \hline y_B \end{pmatrix}$

A_{TL} is 0×0 and t_T, y_T have 0 elements

while $n(A_{BR}) > 0$

$\begin{pmatrix} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{pmatrix} \rightarrow \left(\begin{array}{cc|c} A_{00} & a_{01} & A_{02} \\ a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array}\right),$

$\begin{pmatrix} t_T \\ \hline t_B \end{pmatrix} \rightarrow \begin{pmatrix} t_0 \\ \tau_1 \\ \hline t_2 \end{pmatrix}, \begin{pmatrix} y_T \\ \hline y_B \end{pmatrix} \rightarrow \begin{pmatrix} y_0 \\ \psi_1 \\ \hline y_2 \end{pmatrix}$

Update $\begin{pmatrix} \psi_1 \\ y_2 \end{pmatrix} := \left(I - \frac{1}{\tau_1} \begin{pmatrix} 1 \\ a_{21} \end{pmatrix} \begin{pmatrix} 1 & a_{21}^H \end{pmatrix} \right) \begin{pmatrix} \psi_1 \\ y_2 \end{pmatrix}$

via the steps

$\omega_1 := (\psi_1 + a_{21}^H y_2)/\tau_1$

$\begin{pmatrix} \psi_1 \\ y_2 \end{pmatrix} := \begin{pmatrix} \psi_1 - \omega_1 \\ y_2 - \omega_1 a_{21} \end{pmatrix}$

$\begin{pmatrix} A_{TL} & A_{TR} \\ \hline A_{BL} & A_{BR} \end{pmatrix} \leftarrow \left(\begin{array}{cc|c} A_{00} & a_{01} & A_{02} \\ a_{10}^T & \alpha_{11} & a_{12}^T \\ \hline A_{20} & a_{21} & A_{22} \end{array}\right),$

$\begin{pmatrix} t_T \\ \hline t_B \end{pmatrix} \leftarrow \begin{pmatrix} t_0 \\ \tau_1 \\ \hline t_2 \end{pmatrix}, \begin{pmatrix} y_T \\ \hline y_B \end{pmatrix} \leftarrow \begin{pmatrix} y_0 \\ \psi_1 \\ \hline y_2 \end{pmatrix}$

endwhile

Figure 3.3.6.1. Algorithm for computing $y := Q^H y (= H_{n-1} \cdots H_0 y)$ given the output from the algorithm HQR_unb_var1.

Homework 3.3.6.1. What is the approximate cost of algorithm in Figure 3.3.6.1 if Q (stored as Householder vectors in A) is $m \times n$.

▼ Solution

The cost of this algorithm can be analyzed as follows: When y_T is of length k, the bulk of the computation is in a dot product with vectors of length $m - k - 1$ (to compute ω_1) and an axpy operation with vectors of length

$m-k-1$ to subsequently update ψ_1 and y_2. Thus, the cost is approximately given by

$$\sum_{k=0}^{n-1} 4(m-k-1) = 4\sum_{k=0}^{n-1} m - 4\sum_{k=0}^{n-1}(k-1) \approx 4mn - 2n^2.$$

Notice that this is much cheaper than forming Q and then multiplying $Q^H y$.